

LOGIC FOUNDRY

LF-SDR X1

Adaptive Reconfigurable Intelligent SDR Baseband Processor

"An Adaptive ASIC-Ready SDR Baseband IP with Runtime Reconfiguration, Hardware Performance Analytics and Intelligent Link Adaptation."

Design Novelty

Instead of building a normal SDR,

Logic Foundry proposes

A Self-Adaptive SDR Baseband Processor

which can

- ✓ Change modulation automatically
- ✓ Monitor channel quality
- ✓ Reconfigure itself
- ✓ Measure communication performance
- ✓ Generate runtime statistics

All completely in RTL.

Major Novel Features

1. Adaptive Modulation Engine

Instead of fixed modulation,

the SDR automatically switches

High BER

↓

Switch to BPSK

↓

Medium BER

↓

Switch to QPSK

↓

Low BER

↓

Switch to 16QAM

using

LF_LINK_ADAPT_CTRL

2. Runtime Reconfigurable SDR

Configuration registers allow changing

Modulation

FFT Size

Packet Length

CRC Polynomial

Scrambler Enable

Interleaver Enable

Pilot Enable

without changing RTL.

Interface

LF_CFG_REG

3. Intelligent Performance Monitor

Instead of only BER,

the hardware continuously measures

BER

PER

Throughput

Latency

CRC Errors

Packets Sent

Packets Received

Retry Count

Module

LF_PERF_MON

4. Hardware Health Monitor

Novel ASIC feature

Continuously reports

FIFO Overflow

FIFO Underflow

Synchronization Loss

Frame Loss

CRC Failure

Invalid Packet

Module

LF_HEALTH_MON

5. Debug Register Bank

Every subsystem reports status

TX Busy

RX Busy

Current Modulation

Current FFT Size

Packet Counter

Error Counter

BER

SNR

Module

LF_STATUS_REG

6. Dynamic Power Saving

Unused modules automatically disabled.

Example

Using BPSK

↓

Disable

QPSK Mapper

16QAM Mapper

64QAM Mapper

↓

Reduce Dynamic Switching

Module

LF_POWER_CTRL

7. Plug-and-Play Architecture

Every block uses identical interfaces

making future upgrades simple.

Tomorrow

QPSK

can become

256QAM

without changing the rest of the system.

ARCHITECTURE

refer SDR_FINAL_ARCH.xlsx

Signal	Name
Clock	lf_clk_i
Reset	lf_rst_n_i
Data Input	lf_data_i
Data Output	lf_data_o
Valid Input	lf_valid_i
Valid Output	lf_valid_o
Ready	lf_ready_o
Start	lf_start_i
Done	lf_done_o
Configuration	lf_cfg_if
Status	lf_status_if
Performance	lf_perf_if
TX Stream	lf_tx_if
RX Stream	lf_rx_if
Channel	lf_channel_if

Member 1 – TX Design Engineer

Modules

LF_CFG_REG

LF_MAIN_CONTROLLER

LF_PKT_GEN

LF_CRC_GEN

LF_SCRAMBLER

LF_INTERLEAVER

LF_MOD_CTRL

LF_BPSK_MOD

LF_QPSK_MOD

LF_QAM16_MOD

LF_PILOT_INSERT

LF_S2P

LF_IFFT

LF_CP_INSERT

LF_TX_FIFO

Deliverables

- RTL
- Unit Testbench
- Simulation
- Documentation

Member 2 – RX Design Engineer

Modules

LF_RX_FIFO

LF_FRAME_DETECT

LF_SYNC

LF_CP_REMOVE

LF_FFT

LF_CHANNEL_EST

LF_EQUALIZER

LF_P2S

LF_DEMOD_CTRL

LF_BPSK_DEMOD

LF_QPSK_DEMOD

LF_QAM16_DEMOD

LF_DEINTERLEAVER

LF_DESCRAMBLER

LF_CRC_CHECK

LF_PACKET_DECODER

Performance Modules

LF_BER_COUNTER

LF_SNR_ESTIMATOR

LF_PACKET_COUNTER

LF_LATENCY_COUNTER

LF_PERF_MON

Deliverables

- RTL
 - BER Validation
 - CRC Validation
 - Performance Metrics
-

Member 3 – Verification & Channel Engineer

Channel Modules

LF_AWGN

LF_DELAY

LF_FREQ_OFFSET

LF_CLOCK_DRIFT

Verification

LF_TOP_TB

LF_DRIVER

LF_MONITOR

LF_SCOREBOARD

LF_REFERENCE_MODEL

LF_ASSERTIONS

LF_FUNCTIONAL_COVERAGE

(idhula basic self testing test bench dha you can even use uvm if you can able to explain it)

Deliverables

- Self-checking Testbench
 - Assertions
 - Functional Coverage
 - BER Validation
 - Regression Results
-

Member 4 – Integration & Physical Design Engineer

Integration

LF_TOP

LF_SYSTEM_BUS

LF_INTERCONNECT

LF_MAIN_CONTROLLER

and ALSO physical design complete flow... no worries each and every one assists eachother

1. LF_CFG_REG (Configuration Register Bank)

What is it?

A bank of programmable registers that stores all system configuration parameters.

Why is it needed?

Instead of hardcoding values in RTL, commercial IPs allow software to configure the hardware.

Configurable Parameters

- Modulation Type
- FFT Size
- Packet Length
- CRC Enable
- Scrambler Enable
- Adaptive Mode Enable
- Power Saving Enable

Inputs

Configuration Bus

Clock

Reset

Outputs

Configuration Signals

2. LF_MAIN_CONTROLLER

What is it?

The master FSM that controls the complete SDR operation.

Why?

Without a controller every module runs independently.

The controller coordinates

Packet Generation

↓

Modulation

↓

Transmission

↓

Reception

↓

CRC Check

↓

Performance Update

FSM

IDLE

↓

CONFIG

↓

TX

↓

CHANNEL

↓

RX

↓

CHECK

↓

UPDATE STATUS

↓

IDLE

3. LF_PKT_GEN

What?

Creates packets from incoming user data.

Why?

Real communication systems transmit packets rather than random bits.

Output

Packet Header

Payload

Packet ID

4. LF_CRC_GEN

What?

Calculates CRC bits.

Why?

Allows receiver to detect transmission errors.

Example

Input

11010101

↓

CRC

1011

↓

Transmit

110101011011

5. LF_SCRAMBLER

What?

Randomizes transmitted bits.

Why?

Avoids long runs of 1s or 0s which are difficult for synchronization.

Example

111111111111

↓

101001101011

6. LF_INTERLEAVER

What?

Rearranges bit order.

Why?

Converts burst errors into random errors.

Burst errors are easier to correct after deinterleaving.

7. LF_LINK_ADAPT_CTRL Novel

What?

Monitors communication quality.

Why?

Automatically selects the best modulation scheme.

Example

BER High

↓

Use BPSK

BER Medium

↓

Use QPSK

BER Low

↓

Use 16QAM

8. LF_MOD_CTRL

What?

Selects modulation block.

Why?

Acts like a multiplexer.

Select

↓

BPSK

or

QPSK

or

16QAM

9. LF_BPSK_MOD

What?

Maps

0 → -1

1 → +1

Why?

Most robust modulation.

10. LF_QPSK_MOD

What?

Maps two bits into one symbol.

Example

00

01

10

11

Why?

Double throughput compared to BPSK.

11. LF_QAM16_MOD

What?

Maps four bits into one symbol.

Why?

Higher throughput.

Used when channel quality is good.

12. LF_PILOT_INSERT

What?

Inserts known reference symbols.

Why?

Receiver estimates channel characteristics using pilot symbols.

13. LF_S2P

What?

Serial to Parallel converter.

Why?

IFFT requires parallel input.

14. LF_IFFT

What?

Converts frequency-domain symbols into time-domain OFDM waveform.

Why?

Core of OFDM transmission.

15. LF_CP_INSERT

What?

Adds Cyclic Prefix.

Why?

Reduces inter-symbol interference.

16. LF_TX_FIFO

What?

Temporary packet storage.

Why?

Prevents transmitter stalls.

17. LF_AWGN

What?

Adds Gaussian-like random noise (simplified for RTL).

Why?

Simulates a noisy wireless channel.

18. LF_DELAY

What?

Introduces transmission delay.

Why?

Simulates propagation delay.

19. LF_FREQ_OFFSET

What?

Applies frequency mismatch.

Why?

Models oscillator differences.

20. LF_CLOCK_DRIFT

What?

Simulates transmitter/receiver clock mismatch.

Why?

Real radios never share a perfect clock.

21. LF_RX_FIFO

What?

Stores received packets.

Why?

Buffers variable incoming data rates.

22. LF_FRAME_DETECT

What?

Detects the start of a valid frame.

Why?

Ensures processing begins at packet boundaries.

23. LF_SYNC

What?

Synchronizes received symbols.

Why?

Correct alignment is required before FFT.

24. LF_CP_REMOVE

What?

Removes Cyclic Prefix.

Why?

Recovers the original OFDM symbol.

25. LF_FFT

What?

Converts time-domain OFDM signal back to frequency-domain symbols.

Why?

Reverse of IFFT.

26. LF_CHANNEL_EST

What?

Estimates channel effects using pilot symbols.

Why?

Allows correction of channel distortion.

27. LF_EQUALIZER

What?

Compensates for channel distortion.

Why?

Improves symbol recovery and reduces BER.

28. LF_P2S

What?

Parallel to Serial converter.

Why?

Converts FFT outputs back into a bit stream.

29. LF_DEMOD_CTRL

What?

Chooses the active demodulator.

Why?

Matches the modulation used at the transmitter.

30. LF_BPSK_DEMOD

What?

Recovers bits from BPSK symbols.

Why?

Reverse operation of BPSK modulation.

31. LF_QPSK_DEMOD

What?

Recovers bit pairs from QPSK symbols.

Why?

Reconstructs transmitted data.

32. LF_QAM16_DEMOD

What?

Recovers four-bit symbols.

Why?

Supports high-throughput operation.

33. LF_DEINTERLEAVER

What?

Restores the original bit order.

Why?

Reverses interleaving before error checking.

34. LF_DESCRAMBLER

What?

Restores original bit sequence.

Why?

Inverse of the scrambler.

35. LF_CRC_CHECK

What?

Recomputes CRC and compares it with the received CRC.

Why?

Detects packet corruption.

36. LF_BER_COUNTER ★ Novel

What?

Compares transmitted bits with received bits.

Why?

Measures communication quality.

Output

BER

Bit Errors

Total Bits

37. LF_SNR_ESTIMATOR ★ Novel

What?

Estimates signal-to-noise ratio.

Why?

Provides a channel quality metric and can support adaptive modulation.

38. LF_PACKET_COUNTER ★ Novel

What?

Counts transmitted, received, and dropped packets.

Why?

Useful for throughput analysis and debugging.

39. LF_LATENCY_COUNTER ★ Novel

What?

Counts clock cycles from packet transmission to successful reception.

Why?

Measures end-to-end communication delay.

40. LF_PERF_MON ★ Novel

What?

Central performance monitoring block.

Why?

Collects metrics from BER, SNR, latency, and packet counters into one interface.

41. LF_STATUS_REG ★ Novel

What?

Stores real-time system status.

Why?

Allows software and testbenches to monitor system state.

Example fields:

Current Modulation

FFT Size

TX Busy

RX Busy

Packet Count

BER

SNR

42. LF_HEALTH_MON ★ Novel

What?

Monitors internal hardware faults.

Why?

Improves robustness and debuggability.

Detects:

- FIFO Overflow
- FIFO Underflow
- Frame Loss
- Synchronization Loss
- CRC Failure
- Invalid Packet

43. LF_POWER_CTRL ★ Novel

What?

Disables inactive hardware blocks to reduce switching activity.

Why?

Demonstrates ASIC-oriented low-power design.

Example:

Using BPSK

↓

Disable

QPSK

16QAM

↓

Lower Dynamic Power

44. LF_TOP

What?

Top-level integration module.

Why?

Connects all submodules into one synthesizable SDR IP.